WorldTrace 航班追蹤與飛行護照系統開發規格書

Flighty / byAir 風格架構整合 · Gemini 3.5 Flash-Lite 機票自動解析 · MapLibre 大圓航線與 SQLite v3 實作指南

1. 核心架構原則與對接規範

本規格書針對 waltwait/WorldTrace 專案架構量身設計，在不破壞現有「真實物理 GPS 迷霧探索」核心原則的前提下，完整導入 Flighty / byAir 風格的航班追蹤、Google AI Studio (`gemini-3.5-flash-lite`) 機票檔案自動解析、大圓航線地圖疊加，以及個人飛行護照（Profile）統計系統。所有功能皆為 App 原生內建功能，不設任何 Pro 付費牆或推銷橫幅。

架構原則	現有 WorldTrace 規範 (CLAUDE.md)	航班追蹤模組對接設計
真實 GPS 鐵則	`points` 與 `fog_tiles` 僅接受經由 `src/gatekeeper/` 驗證的真實物理 GPS 座標，拒絕模擬位置與外部軌跡匯入。	航班資料獨立儲存於 SQLite 的 `flights` 與 `flight_docs` 表，**絕不寫入** `points` 或 `fog_tiles`，確保地面探索面積 100% 源自真實移動。
Ports & Adapters	資料庫透過 src/store/driver.ts 的 `SqlDriver` 抽象化；雲端透過 `DriveTransport` 抽象化。	`src/store/flights.ts` 完全使用 `SqlDriver` 介面；外部航空 API 與 Gemini AI 解析亦透過 Port 封裝，支援在 Vitest 中以 `node:sqlite` 執行單元測試。
導覽與地圖留存	App.tsx 使用 `pagesSlide` 與 `openedTabs` 水平滑動與延遲掛載，避免 MapLibre 重複重建。	新增的 `FlightsScreen` 直接接入 `openedTabs` 機制；大圓航線作為獨立 `<GeoJSONSource id="flight-arcs">` 圖層疊加於 MapLibre 實例上。
雲端備份整合	src/cloud/ 定期將本地 SQLite 資料庫快照備份至 Google Drive。	升級 src/store/schema.ts 至 `SCHEMA_VERSION = 3`，所有航班紀錄與統計自動隨現有 Google Drive 備份機制同步與還原。

2. 專案目錄與新增/修改檔案清單

建議在 src/ 中擴充以下模組，維持既有領域邏輯與 UI 分離的設計：

```
WorldTrace/
├── App.tsx                          # [修改] 更新底部導覽列為 Flighty 膠囊風格，掛載 FlightsScreen 與航班詳情抽屜
├── package.json                     # [修改] 新增 expo-document-picker 與 expo-image-picker
└── src/
    ├── flights/                     # [新增目錄] 航班核心領域邏輯 (純函式與 Port 介面，支援 Vitest 測試)
    │   ├── types.ts                 # 航班、機場、AI 解析結果與狀態列舉型別定義
    │   ├── airports.ts              # 離線 IATA 機場索引 (經緯度、城市、國旗、IANA 時區) 與時差計
```

```
算
    │   ├── greatCircle.ts              # Haversine 飛行里程計算與球面大圓航線 GeoJSON 生成器
    │   ├── geminiParser.ts             # 串接 gemini-3.5-flash-lite 結構化解析 PDF / 登機證圖片
    │   ├── flightStatusApi.ts          # 航空狀態追蹤 Port 與 AeroDataBox / FlightAware Adapter
    │   └── flightStats.ts              # 計算個人飛行護照統計 (總趟數、時數、里程、國家數、印章)
    ├── store/
    │   ├── schema.ts                   # [修改] SCHEMA_VERSION 升至 3，新增 flights 與
flight_docs 資料表
    │   └── flights.ts                  # [新增] 封裝航班 CRUD、Active/Finished 篩選與狀態更新 SQL
    └── ui/
        ├── theme.ts                    # [修改] 擴充 Flighty 純黑底、深灰卡片與 #facc15 機場指示牌黃
色 Token
        ├── MapScreen.tsx               # [修改] 新增 flight-arcs 大圓航線圖層與圖層顯示開關 (迷霧 /
航線 / 雙層)
        ├── FlightsScreen.tsx           # [新增] My Flights 列表頁 (Active/Finished 切換、倒數天
數、快捷列、+ FAB)
        ├── FlightDetailSheet.tsx       # [新增] 航班詳情頁 (航線地圖、黃色登機門/轉盤牌、PNR 複製放
大、時區卡)
        ├── AddFlightSheet.tsx          # [新增] 新增航班彈窗 (AI 機票檔案上傳預覽 + 手動輸入表單)
        └── StatsScreen.tsx             # [修改] 升級為純淨版 Flighty Profile (藍紫漸層飛行護照卡 +
地面迷霧探索統計)
```

環境依賴安裝與設定

專案已內建 expo-file-system 與 expo-sqlite。請安裝相容於 Expo SDK 57 的檔案選取套件（expo-document-picker 與 expo-image-picker），並在 `.env` 設定 API 金鑰：

```
npx expo install expo-document-picker expo-image-picker

# .env 設定
EXPO_PUBLIC_GEMINI_API_KEY=your_ai_studio_api_key_here
EXPO_PUBLIC_FLIGHT_API_KEY=your_aerodatabox_or_flightaware_key_here
```

3. 資料庫 Schema 升級規格 (SCHEMA_VERSION = 3)

將 src/store/schema.ts 中的 `SCHEMA_VERSION` 由 `2` 升至 `3`，並加入以下遷移 SQL 與 `src/store/flights.ts` 存取函式：

```
// src/store/schema.ts (V3 Migration SQL)
export const SCHEMA_VERSION = 3;

export const MIGRATION_V3_SQL = `
CREATE TABLE IF NOT EXISTS flights (
  id TEXT PRIMARY KEY,
  flight_number TEXT NOT NULL,          -- 例: "CI501"
  airline_code TEXT NOT NULL,           -- 例: "CI"
  airline_name TEXT NOT NULL,           -- 例: "China Airlines"
  alliance TEXT,                        -- 例: "SkyTeam" | "Star Alliance" | "oneworld"
  dep_iata TEXT NOT NULL,               -- 例: "TPE"
  arr_iata TEXT NOT NULL,               -- 例: "PVG"
  dep_scheduled_utc INTEGER NOT NULL,   -- 表定起飛時間 (Unix epoch ms)
  dep_estimated_utc INTEGER,            -- 預計/實際起飛時間 (Unix epoch ms)
  arr_scheduled_utc INTEGER NOT NULL,   -- 表定抵達時間 (Unix epoch ms)
  arr_estimated_utc INTEGER,            -- 預計/實際抵達時間 (Unix epoch ms)
  dep_tz TEXT NOT NULL,                 -- 例: "Asia/Taipei"
  arr_tz TEXT NOT NULL,                 -- 例: "Asia/Shanghai"
  dep_terminal TEXT,                    -- 例: "2"
```

```
  arr_terminal TEXT,                    -- 例: "1"
  dep_gate TEXT,                        -- 例: "D6"
  arr_gate TEXT,                        -- 例: "18"
  baggage_belt TEXT,                    -- 例: "7"
  status TEXT NOT NULL DEFAULT 'SCHEDULED', -- SCHEDULED | BOARDING | IN_AIR | LANDED |
DELAYED | CANCELLED
  distance_miles REAL NOT NULL,         -- 例: 420.0
  duration_minutes INTEGER NOT NULL,    -- 例: 125
  seat TEXT,                            -- 例: "24A"
  cabin_class TEXT,                     -- Economy | Premium Economy | Business | First
  pnr TEXT,                             -- 訂位代號，例: "7X9K2M"
  notes TEXT,                           -- 個人備註
  notifications_enabled INTEGER NOT NULL DEFAULT 0,
  is_finished INTEGER NOT NULL DEFAULT 0,
  last_synced_at INTEGER,
  created_at INTEGER NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_flights_dep_time ON flights(dep_scheduled_utc DESC);
CREATE INDEX IF NOT EXISTS idx_flights_finished ON flights(is_finished, dep_scheduled_utc
ASC);

CREATE TABLE IF NOT EXISTS flight_docs (
  id TEXT PRIMARY KEY,
  flight_id TEXT NOT NULL REFERENCES flights(id) ON DELETE CASCADE,
  file_name TEXT NOT NULL,
  local_uri TEXT NOT NULL,
  mime_type TEXT NOT NULL,
  created_at INTEGER NOT NULL
);
`;
```

4. 核心領域模組實作規格 (src/flights/)

4.1 Gemini 3.5 Flash-Lite 機票檔案自動解析 (src/flights/geminiParser.ts)

呼叫 Google AI Studio Gemini 3.5 Flash-Lite (gemini-3.5-flash-lite)，支援直接傳入 PDF 電子機票或登機證截圖，並透過 Structured Outputs (responseSchema) 確保回傳嚴格符合型別的航段陣列：

```
// src/flights/geminiParser.ts
import * as FileSystem from 'expo-file-system';

export interface GeminiExtractedLeg {
  airlineCode: string;
  airlineName: string;
  alliance?: string;
  flightNumber: string;
  depIata: string;
  arrIata: string;
  depScheduledIso: string;
  arrScheduledIso: string;
  depTerminal?: string;
  arrTerminal?: string;
  depGate?: string;
  seat?: string;
  pnr?: string;
  cabinClass?: string;
```

```
}

export async function parseTicketFileWithGemini(
  fileUri: string,
  mimeType: string,
  apiKey: string
): Promise<GeminiExtractedLeg[]> {
  const base64Data = await FileSystem.readAsStringAsync(fileUri, {
    encoding: FileSystem.EncodingType.Base64,
  });

  const url = `https://generativelanguage.googleapis.com/v1beta/models/gemini-3.5-flash-
lite:generateContent?key=${encodeURIComponent(apiKey)}`;

  const response = await fetch(url, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      contents: [{
        parts: [
          { inline_data: { mime_type: mimeType, data: base64Data } },
          { text: '請從這份機票、行程單或登機證中擷取所有航段資料。機場請轉為大寫 3 碼 IATA 代碼，起降時間請
含當地時區輸出為標準 ISO 8601 字串。' }
        ]
      }],
      generationConfig: {
        responseMimeType: 'application/json',
        responseSchema: {
          type: 'ARRAY',
          items: {
            type: 'OBJECT',
            properties: {
              airlineCode: { type: 'STRING' },
              airlineName: { type: 'STRING' },
              alliance: { type: 'STRING', nullable: true },
              flightNumber: { type: 'STRING' },
              depIata: { type: 'STRING' },
              arrIata: { type: 'STRING' },
              depScheduledIso: { type: 'STRING' },
              arrScheduledIso: { type: 'STRING' },
              depTerminal: { type: 'STRING', nullable: true },
              arrTerminal: { type: 'STRING', nullable: true },
              depGate: { type: 'STRING', nullable: true },
              seat: { type: 'STRING', nullable: true },
              pnr: { type: 'STRING', nullable: true },
              cabinClass: { type: 'STRING', nullable: true }
            },
            required: ['airlineCode', 'airlineName', 'flightNumber', 'depIata', 'arrIata',
'depScheduledIso', 'arrScheduledIso']
          }
        }
      }
    })
  });

  if (!response.ok) throw new Error(`Gemini 解析失敗: ${response.status}`);
  const data = await response.json();
  return JSON.parse(data.candidates[0].content.parts[0].text) as GeminiExtractedLeg[];
}
```

4.2 離線機場庫、大圓航線插值與時差計算 (airports.ts & greatCircle.ts)

由開源的 OurAirports 資料集 萃取全球商業機場的 IATA、經緯度、國旗 Emoji 與 UTC 時區偏移量，支援離線計算球面大圓弧線（Slerp 插值生成 64 段 GeoJSON `LineString`）與抵達時區變化文字（如 `No timezone change · Taipei and Shanghai are in the same timezone [UTC+8]`）。

4.3 航空數據 API 選型與智慧分級輪詢策略 (flightStatusApi.ts)

為兼顧即時性與免費/平價 API 額度，建議以 AeroDataBox API 或 FlightAware AeroAPI 作為數據源，並依距離起飛時間實施三階段輪詢：

- **起飛前大於 48 小時**：僅在使用者手動下拉更新或每 24 小時開啟 App 時同步一次班表異動。
- **起飛前 24 至 48 小時（網路報到開放期）**：每 4 小時同步一次，鎖定起降航廈（Terminal）與預排登機門。
- **起飛前 6 小時內至降落後 1 小時**：每 15 分鐘拉取最新登機門（Gate）、準點狀態（On time / Delayed）與抵達行李轉盤（Baggage Belt）。

5. UI/UX 設計系統與 React Native 元件規格 (src/ui/)

升級 src/ui/theme.ts 為 Flighty / byAir 風格的 OLED 純黑與機場指示牌黃色視覺系統：

Token 名稱	現有 WorldTrace 值	Flighty 風格升級值	應用位置
`background`	`#070b16`	`#000000`	My Flights 與抽屜底層 OLED 純黑背景
`surfaceSolid`	`#0c1223`	`#161618`	航班卡片、倒數狀態卡與 Details 容器表面
`accent`	`#5b9cff`	`#6366f1`	`Active 1` 膠囊、右下角 `+` FAB 與底部導覽高亮
`live`	`#4ade80`	`#22c55e`	`08:45 AM` 起降時間與 `On time` 準點綠字
`airportSignBg / Text`	新增 Token	`#facc15` / `#09090b`	`↗ D6` 起飛登機門、`↘ 18` 抵達門、🧳 `7` 行李轉盤黃牌
`passportGradient`	新增 Token	`#2b2768` → `#4845b8`	Profile 個人飛行護照統計卡藍紫漸層底色

四大畫面實作要點

- **My Flights 列表頁 (`src/ui/FlightsScreen.tsx`)**：配置 `Active` / `Finished` 膠囊篩選器、依日期分組的深灰航班卡片（含中央大字體倒數 `56d`）、卡片下方的獨立懸浮快捷列（通知開關、分享、更多選單），以及右下角 `#6366f1` 圓角新增按鈕。
- **Flight Detail 航班詳情頁 (`src/ui/FlightDetailSheet.tsx`)**：上半部為 MapLibre 大圓航線圖（提供「疊加地面迷霧」按鈕），下半部圓角抽屜整合倒數狀態卡、`#facc15` 機場指示牌黃色徽章（點擊可手動修正登機

門與轉盤）、Details 列表（含 PNR 一鍵複製與全螢幕放大出示、座位編輯、備註、紫色圓章 `Travel Docs` 附件預覽）及 On Arrival 跨時區自動對比卡。

- **純淨版 Profile 飛行護照與地面探索頁 (`src/ui/StatsScreen.tsx`)**：完全移除任何 Pro 方案橫幅，上方呈現全航線地圖與個人頭像，中段為藍紫漸層飛行護照卡（`FLIGHTS`、`HOURS`、`MI`、`COUNTRIES`、`Flight stats ›` 與國家數位印章彈窗），下方無縫銜接 WorldTrace 原有的「已探索面積」、「地球探索佔比」、「探索磚格數（z16）」與 Google Drive 備份按鈕。
- **MapLibre 雙層地圖整合 (`src/ui/MapScreen.tsx`)**：在原有的 `fog` 與 `day-track` 圖層之上加入 `flight-arcs` 圖層，並於頂部提供「雙層 / 僅迷霧 / 僅航線」快速切換膠囊。

6. 分階段開發與測試驗收計畫

開發階段	核心交付檔案	驗收標準與測試方式
Phase 1 (Day 1)	`src/store/schema.ts` (v3)、`src/store/flights.ts`、`src/flights/airports.ts`、`greatCircle.ts`、`flightStats.ts`	執行 `npm test`（Vitest + `node:sqlite`），驗證航班 CRUD、跨時區計算與大圓距離單元測試 100% 通過，且不影響既有 480 項迷霧測試。
Phase 2 (Day 2)	`src/flights/geminiParser.ts`、`src/ui/AddFlightSheet.tsx`	上傳 PDF 電子機票與登機證截圖，驗證 `gemini-3.5-flash-lite` 回傳結構化 JSON 並成功寫入 `flights` 與 `flight_docs`。
Phase 3 (Day 3–4)	`src/ui/theme.ts`、`FlightsScreen.tsx`、`FlightDetailSheet.tsx`、`StatsScreen.tsx`、`MapScreen.tsx`	驗證 Flighty 風格卡片、黃色機場指示牌、PNR 複製放大、藍紫漸層護照卡與 MapLibre 航線弧線渲染流暢度。
Phase 4 (Day 5)	`src/flights/flightStatusApi.ts`、Google Drive 備份驗證	驗證分級輪詢更新登機門/轉盤狀態，並測試 Google Drive 雲端備份與還原是否完整保留所有航班紀錄。